

School of Science, Computing and Engineering Technologies
Swinburne University of Technology
SWE30003 Software Architectures and Design
Tutorial Week 5
(Software Requirements)

Please carefully read the brief description of an “Automated Attendance and Access Control” system given below. Form groups three to four students and discuss the following questions:

1. Identify the goal(s) and main objective(s) of the described system.

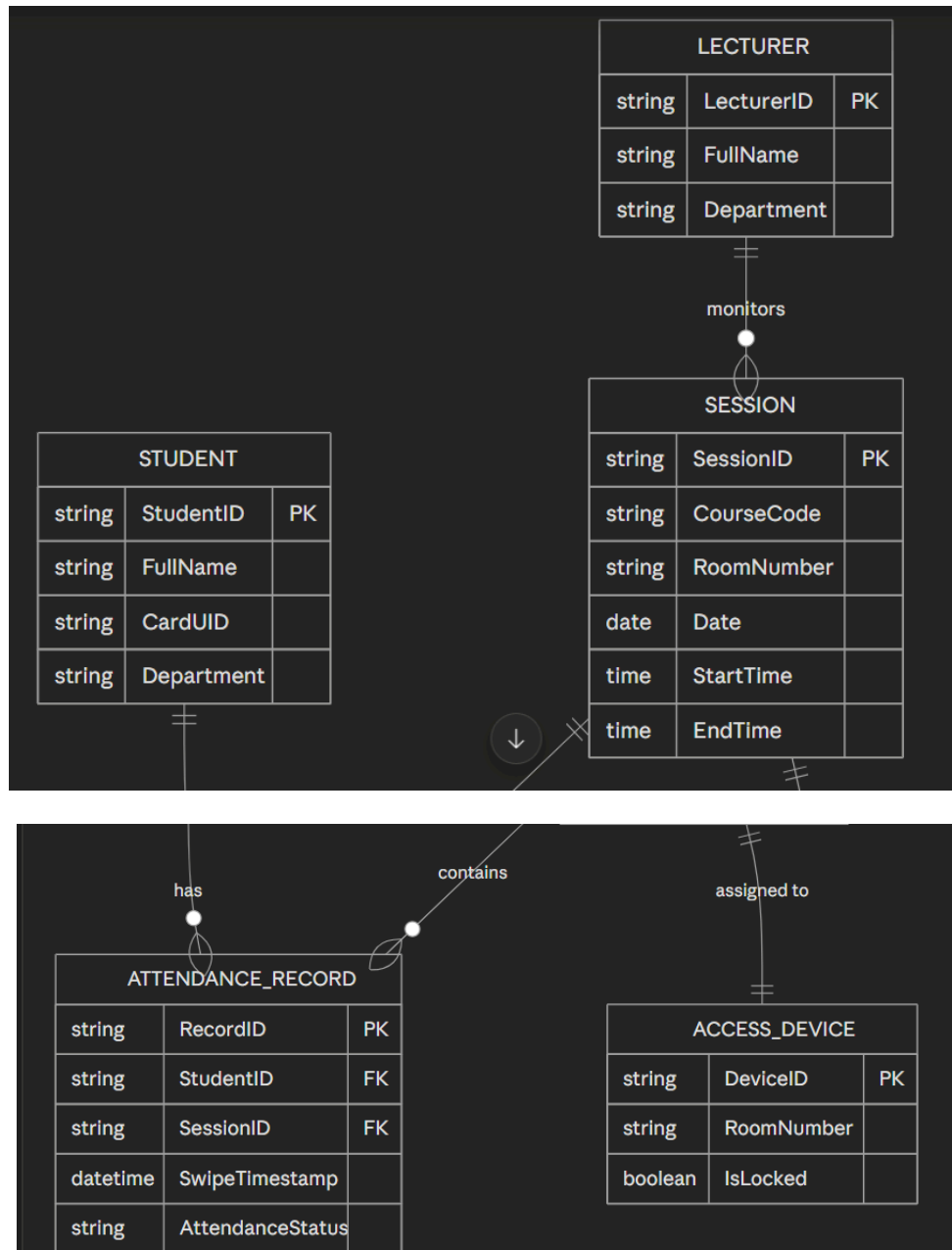
=> The main goal of the system is to modernize how Swinburne tracks attendance due to the transition from manual paper sign-ins to a rapid card-swipe system. Therefore, the system also aims to stop "proxy cheating", particularly when a student signs a paper for an absent friend, and lay the groundwork to secure campus doors using the same cards later on.

Additionally, the system must let hundreds of students swipe their cards rapidly at the door so hallways do not get crowded. Notably, it must automatically create instant, complete digital attendance lists for every lecture and lab for teachers. Finally, the software must be built flexibly so it can connect to electronic door locks in the future without needing to be completely remade.

2. Identify the domain (data) model for the desired system.

=> Main Vocabulary: Student, Lecturer, Session, Attendance Record, and AccessDevice.

- Student: Showcases the university students and contains attributes including StudentID, FullName, CardUID (for rapid swiping), and Department.
- Lecturer: Represents the academic staff monitoring the session, containing LecturerID, FullName, and Department.
- Session: Emphasizes a specific class instance in a timeslot include *SessionID, CourseCode, RoomNumber, Date, StartTime, and EndTime*.
- AttendanceRecord: The transactional entity bridging Students and Sessions (*RecordID, StudentID, SessionID, SwipeTimestamp, and AttendanceStatus Present/Late/Absent*).
- AccessDevice (Future-Proof): Represents the electronic door locks. Contains *DeviceID, RoomNumber, and IsLocked*



3. Identify and *briefly* describe all user tasks that such a system must be able to support.

NOTE: most of the required tasks are not actually mentioned in the brief description but are implied by the purpose of the system.

=> **Student Tasks:**

- **Swipe Card for Attendance:** The high-frequency task where students tap their smart cards at the classroom reader upon entry to log their presence instantly.
- **View Personal Attendance Records:** Students log into their portal/app dashboard to check their attendance history, current attendance rates, and monitor any "Late" or "Absent" flags.

- Submit Attendance Dispute/Exemption Requests: In cases of medical leave or technical reader errors, students must be able to upload supporting digital documents (e.g., medical certificates) to justify an absence.

=> **Lecture Tasks:**

- Monitor Live Session Attendance: Lecturers access a real-time console during class to view the headcount, monitor active student check-ins, and ensure no "proxy cheating" is physically occurring.
- Manually Override Attendance Status: Lecturers must have the authority to manually alter a student's status (e.g., changing "Absent" to "Present" if a student forgot their card but is physically in class).
- Review and Approve Exemption Tickets: Lecturers or academic staff review submitted student absence justifications and approve them to automatically update the official registry.
- Export Attendance Reports: Generate and download comprehensive session or semester-long attendance lists for grading, compliance, or university record-keeping.

=> **System Administrator Tasks:**

- Manage Student/Lecturer Profiles and Card UUIDs: Register new users, issue or update smart cards, and link a card's unique hardware ID (UUID) to a specific student profile.
- Synchronize Master Class Timetables: Import daily or weekly schedules, room allocations, and student enrollment lists from the university's central database to prime the localized classroom readers.
- Configure Access Control Policies (Future-Proof): Program permissions for electronic door locks, defining which student/lecturer cards can unlock specific rooms during designated timeslots.

4. Given the users tasks in the previous step as well as any domain knowledge you have, identify and formulate three to five *different* non-functional requirements/quality attributes for the system. Also, discuss how you would go about *verifying* these quality attributes.

=> Performance (Throughput and Latency):

- Requirement: The card-swipe reader and backend system must process a student's check-in transaction within short time, specifically 0.5 seconds of contact. In addition, the system must natively handle continuous, high-volume traffic of up to 200 concurrent read operations per minute at a single entrance to eliminate hallway congestion.
- Verification Strategy: (Load & Stress Testing): Use performance testing tools like Apache JMeter or Gatling to simulate hundreds of concurrent card-swipe API requests hitting the server simultaneously. Monitor the server response time and verify that the 95th percentile latency remains strictly under 500ms under peak load conditions.

=> Security and Data Integrity:..

- Requirement: To eradicate "proxy cheating" and secure physical spaces, the system must enforce zero-tolerance data integrity rules. Attendance logs must be unalterable once verified, and duplicate swipes from the same Card

UID within a 5-minute window for the same session must be flagged or rejected.

- Verification Strategy (Security & Constraint Testing): Execute boundary-value and penetration test scripts where a single Card UID attempts double-booking or rapid sequential swipes across different readers. Conduct database audit log reviews to ensure that created **AttendanceRecords** cannot be updated or deleted by unauthorized user roles (enforcing strict RBAC - Role-Based Access Control).

=> Extensibility and Modularity

- Requirement: The system infrastructure must decouple the core attendance tracking business logic from the physical hardware layer. Adding new features, such as integrating electronic door locks, must be achievable via modular API endpoints without altering more than 5% of the existing codebase or causing system-wide downtime.
- Verification Strategy (Architecture Review & Dependency Injection Testing): Perform static code analysis and architectural dependency checks. Use mock hardware interfaces to simulate electronic lock signals, verifying that the domain layer can broadcast "unlock" directives via an event-driven mechanism without needing to recompile the attendance management module.

=> Availability and Fault Tolerance:

- Requirement: The student-facing card-swipe interface must maintain an uptime of 99.5% during operating university hours. In the event of a localized network failure, the classroom readers must support offline logging capabilities, retaining data safely until the connection is restored.
- Verification Strategy (Chaos Engineering & Failover Testing): Conduct simulated network disconnects during a live test environment session. Verify that the reader buffers the swipe events locally and seamlessly synchronizes the data with the primary server plane once the connection is re-established, without dropping any records.

5. Are there any issues in regards to this system that you have not identified in (2) and (3), but are of concern to you in the given environment?

- Hallway Crowding: Hundreds of Swinburne students enter big lecture halls at the exact same time. If the card reader takes too long to process a swipe, it will create long lines and make students late.
- No Internet (Offline Mode): If the school's Wi-Fi or network goes down temporarily, the card readers must still work and save swipes locally so students do not get locked out or missed.
- Card Sharing Loophole: While the system stops paper cheating, students can easily hand their plastic cards to a friend to swipe them into class. The system needs a way to handle this new loophole.
- Privacy Concerns: Tracking exactly when and where a student enters a room creates privacy issues. The school must secure this data so only authorized people can see it.

- Wasted Money on Hardware: Since the school wants to add door access later, the card readers bought today must be compatible with electronic door locks tomorrow, or the school will waste money replacing them.

6. (*Optional*) Think about possible solutions to the software side of the system (you can ignore any hardware issues etc. for this question).

=> CQRS Architecture (Command Query Responsibility Segregation)

- Solution: Separate data write pipelines (swiping cards to log attendance) from data read pipelines (lecturers viewing dashboards or exporting reports).
- Benefit: Ensures that high-burst traffic from hundreds of students swiping at the door simultaneously will never lock the database or slow down the web portals.

=> Edge Caching & Offline Synchronization (Offline-First)

- Solution: Swipe data is temporarily stored and validated inside a lightweight local database (like SQLite or Redis) at the classroom level before being asynchronously synced to the cloud back-end.
- Benefit: Eliminates network latency during card-swiping. Even if the university's network drops, students can still check in normally, and data will auto-sync once restored.

=> Event-Driven Architecture (For Future Proofing)

- Solution: Upon a successful check-in, the core system broadcasts an asynchronous event (e.g., **AttendanceLogged**) through a message broker (like RabbitMQ or MQTT).
- Benefit: Perfectly addresses future extensibility. When electronic door locks are introduced later, a small separate service can just listen to this event to unlock the physical door, keeping the core attendance code completely untouched.

=> API Gateway & Role-Based Access Control (RBAC)

- Solution: Implement a centralized API Gateway to route all traffic, enforcing strict token authentication (JWT) to control data access plane permissions based on user roles.
- Benefit: Secures system boundaries—classroom readers can only call the swipe-logging endpoint, students can only view their own history, and only verified lecturers can override records or export reports.

7. Any questions from weekly submissions? => Currently not

Automated Attendance and Access Control System

Presently, Swinburne University of Technology tracks student attendance, where required, by physical signature on the attendance paper circulated during lectures, tutorials, or labs. This is not only time consuming, but also not effective as a student may ask someone to sign on their behalf. Therefore, the university would like to adopt a *swipe-card based* attendance and access control system. Initially, this system should be used to track the attendance of students in lectures, tutorials and/or laboratory sessions. Such a system is expected to reduce manual processing in order to get (more) complete attendance records for individual classes. At a later stage, the system should also be used for door access to facilities in the campus such as computer labs, project rooms and others.

As a result, you are asked to *carefully* elicit and specify the requirements for such an “Automated Attendance and Access Control” system.